



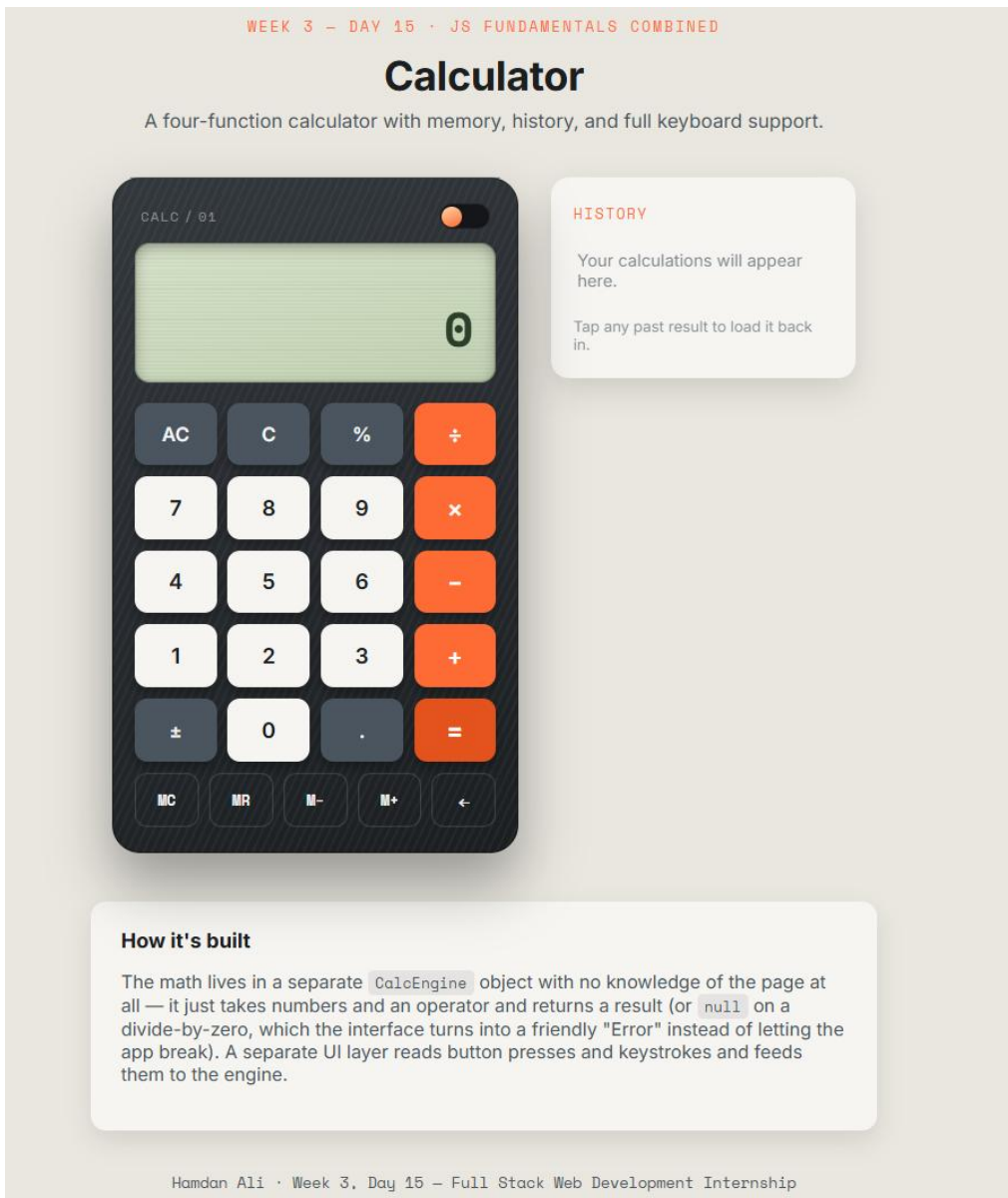
NAME : HAMDAN ALI

WEEK 3 : Day 15

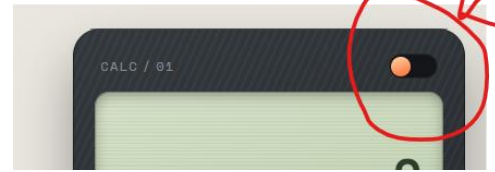
DAY 15

Build a small calculator app using JavaScript (HTML + CSS + JS).

Task 15 : Screenshots Of Output



You can change the UI color And background By just clicking the this button:



RESULT:

WEEK 3 - DAY 15 · JS FUNDAMENTALS COMBINED

Calculator

A four-function calculator with memory, history, and full keyboard support.

CALC / 01

0

AC

C

%

÷

7

8

9

×

4

5

6

-

1

2

3

+

±

0

.

=

MC

MR

M-

M+

←

HISTORY

Your calculations will appear here.

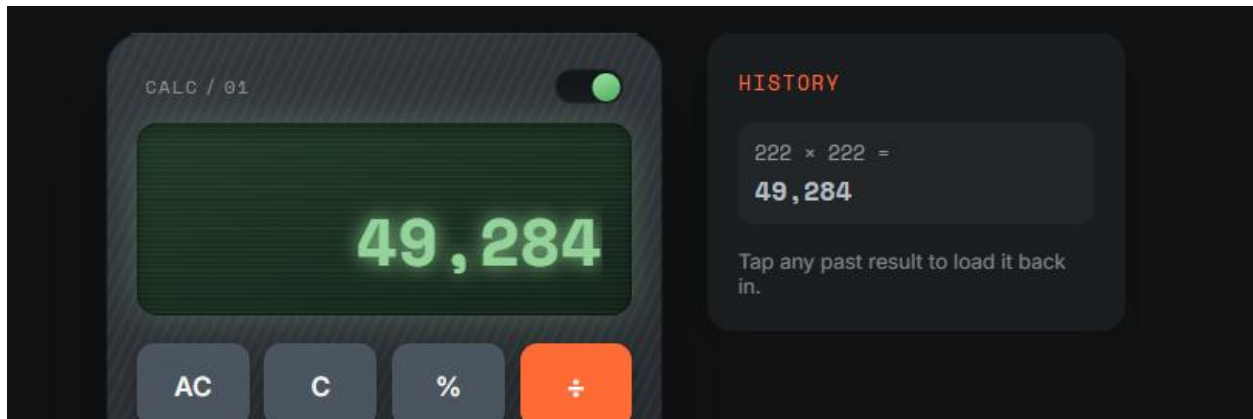
Tap any past result to load it back in.

How it's built

The math lives in a separate `CalcEngine` object with no knowledge of the page at all — it just takes numbers and an operator and returns a result (or `null` on a divide-by-zero, which the interface turns into a friendly "Error" instead of letting the app break). A separate UI layer reads button presses and keystrokes and feeds them to the engine.

Hamdan Ali · Week 3, Day 15 - Full Stack Web Development Internship

After Giving input the result will store in history :



EXPLANATION

DAY 15 — JS FUNDAMENTALS COMBINED

Building a Calculator, Start to Finish

Hamdan Ali · Week 3 · Full Stack Web Development Internship

What I Was Going For

Day 15 was marked as a project rather than a single task, so I treated it like one. Before I opened the editor I sat down and worked out what I actually wanted this thing to do: the four basic operations with proper chaining, memory buttons, a history of past calculations, full keyboard support, and it had to handle bad input — like dividing by zero — without breaking. I also didn't want it to look like the calculator you get from typing "JS calculator tutorial" into YouTube, so the look of it got as much thought as the logic did.

Once I had that list, the build itself came together in two parts: the actual math, and everything that connects that math to the screen.

Splitting the Math From the Interface

I organized the file into two clearly separated halves. The first, CalcEngine, is a plain object that does arithmetic and nothing else — it never once touches document. It takes numbers and an operator and hands back a result:

```
compute(first, second, operator) {  
  switch (operator) {  
    case '+': return this.roundClean(first + second);  
    case '÷':  
      if (second === 0) return null;  
      return this.roundClean(first / second);  
    // ...  
  }  
}
```

I kept it this deliberately separate because it means the actual math could be tested, reused, or dropped into a totally different interface without touching a single line — which is really the point of separating logic from presentation in any real application, not just a calculator.

The second half of the file is everything that touches the page: reading button clicks and key presses, calling into CalcEngine, and writing whatever comes back onto the screen.

Chained Calculations and Why They're Trickier Than They Look

Typing $4 + 3 + 2 =$ and expecting 9 sounds obvious until you try to write it. The engine has to notice that a second $+$ was pressed while a calculation was already waiting, resolve that first part ($4 + 3 = 7$), and only then start waiting for the next number:

```
chooseOperator(nextOperator, currentValue) {  
  if (this.firstOperand === null) {  
    this.firstOperand = currentValue; } else if (this.operator && !this.waitingForSecondOperand)  
  { const result = this.compute(this.firstOperand, currentValue, this.operator);  
    this.firstOperand = result; }  
  this.waitingForSecondOperand = true;  
  this.operator = nextOperator;  
  return this.firstOperand;  
}
```

I actually caught a bug here while testing it — calling this function directly with hand-picked values gave the wrong answer, because in the real app, typing a new digit is what flips `waitForSecondOperand` back to `false` in between operator presses. Testing it in an order that skipped that step gave a false result, which was a good reminder that logic needs to be tested the way it's actually going to be used, not just in whatever order seems convenient at the time.

Handling Divide-by-Zero Without Crashing

JavaScript doesn't throw an error for $5 / 0$ — it just quietly returns `Infinity`, which would show up on the display as an ugly, confusing number. Instead, `compute()` returns `null` for that specific case, and the interface treats `null` as one clear signal to show "Error" and reset after a short pause, rather than checking for `Infinity`, `-Infinity`, and `NaN` separately in multiple places:

```
if (result === null) {  
  showError();  
  return;  
}
```

Avoiding the Classic $0.1 + 0.2$ Bug

Most calculator tutorials skip this, and it shows the moment someone tries $0.1 + 0.2$ and gets `0.30000000000000004` instead of `0.3` — a well-known quirk of how computers store decimal numbers. I added a small rounding helper that every result passes through before it's returned, which clears out that floating-point noise without cutting off real precision:

```
roundClean(value) {  
  return Math.round((value + Number.EPSILON) * 1e10) / 1e10;  
}
```

Memory, History, and Full Keyboard Support

Beyond the core four operations, the calculator has a memory register (`M+`, `M-`, `MR`, `MC`) stored directly on the engine, and a history panel that keeps the last eight calculations, clickable to load a past result straight back into the display. Every one of these also works from the keyboard, not just the mouse — digits, the four operators, `Enter` for equals, `Escape` to clear, `Backspace` to delete a

digit — through one keydown listener mapped to the exact same functions the on-screen buttons call, so there's no second, slightly different code path to keep in sync.

The Design — "The Precision Instrument"

For the interface, I deliberately avoided the generic flat-card calculator look most tutorials produce. The idea I built toward instead: a calculator is physically a measurement instrument, so the page treats it like one — a brushed-graphite chassis, a recessed LCD-style display with a soft phosphor glow and a scanline texture, and keys with a real press animation rather than just a color change on click.

The light/dark toggle doesn't just invert the whole page — the metal chassis stays a constant dark graphite in both modes, because a real device's casing doesn't change color when you flip a display setting. What actually changes is the LCD backlight (pale sage-green by day, glowing green-on-black by night) and the keycap material, which is closer to how an actual instrument's day/night mode would behave.

A Few Notes on What I Took From This

- Writing out the full feature list before starting made the scope decisions — what's in, what's deliberately left out — clear from the start instead of something I was guessing at halfway through the build.
- Separating the calculation engine from the DOM code made the trickiest logic (chained operations, divide-by-zero) far easier to reason about and test on its own.
- A bug only showed up when I tested the code in an order that didn't match how a real user would actually use it — a good reminder that how you test something matters as much as whether you test it.
- Rounding floating-point results is a small detail almost every beginner calculator tutorial skips, and it's one of the first things that makes an app feel unfinished if it's missing.

References Used

I used the linked Tutorialspoint walkthrough for the general shape of a real-time HTML/CSS/JS calculator, and the linked video to see the interaction logic built step by step, then extended both well past their scope for memory, history, keyboard support, and the visual design.



Article: [tutorialspoint.com — real-time calculator](https://tutorialspoint.com/real-time-calculator/)

Video: youtube.com/watch?v=TXCj39dGLIY

Files Submitted

- index.html — semantic structure for the calculator, history panel, and theme toggle.
- style.css — the full "Precision Instrument" design system, light and dark themes.
- script.js — CalcEngine (pure logic) and the UI controller that connects it to the page.

THANK YOU!